Cooker - find what you can do, how to start, and why

NISR 2026 Big Data Hackathon - Track 3: Open Innovation

Cooker helps young Rwandans discover work paths that fit them, then gives small tested first steps (20 min/day, free) instead of a job-title guess. It runs in the browser in Kinyarwanda, English, French and Swahili (Arabic and Luganda fall back to English until reviewed), works offline as a PWA, and needs no typing.

Live demo: https://cooker-public.vercel.app

Repository: https://github.com/map-boy/cooker-nisr

How it works

1. Language, consent, 8 context taps (age band, situation, data/device, money, time).
2. Joy map (what you really do) + adaptive yes/no/forced-choice cards; stops by rule when the answer is stable.
3. 3 short mini-games (focus, reasoning, memory) add behavioural evidence.
4. Ranking = interest + style + motives + evidence + reality, weighted by life stage, **plus a NISR labour-demand factor** (LFS Q3 2026: sector share of workers and one-year job change).
5. The result explains "because you said..." and starts a 5-station Journey (watch, copy, make, show, ask) with "not yet" barrier fixes.

Full algorithm: COOKER.md. Honest limits: weights are expert starting values, not validated; the path-to-sector mapping is our assumption (see nisr/docs).

Run

```
npm install
npm run dev
npm run typecheck && npm run lint:content && npm run sim && npm run build
```

The engine (packages/engine) is pure and deterministic (seeded). tools/simulate.ts runs persona tests. Games accept ?seed=123 for repeatable sequences.

NISR data used

Dataset	Used for
Labour Force Survey Q3 2026 tables and August 2026 report	sector share of workers, one-year job change (demand factor), youth/adult and education indicators
LFS 2022 annual report	context and definitions

Dataset	Used for
EICV7 documentation, FinScope 2024 report	planned district-level and access context (not yet in ranking)

Processed tables: nisr/data/processed. Build scripts: nisr/*.py. Checked figures: nisr/docs/verified_facts.md.

AI-tools disclosure

Code, documentation and data-parsing scripts were drafted with Claude (Anthropic) and run, tested and reviewed by the team. Kinyarwanda, French and Swahili texts are drafts pending native-speaker review.

Team

Team PK: Mugisha Alain Paisible (ULK, Data Science).

PROBLEM STATEMENT - Cooker Pathfinder (numbers: NISR Labour Force Survey, Aug 2026 / Q3)

1. The problem 1,038,707 young Rwandans (27.5% of 16-30 year-olds) are not in employment, education or training. For young women it is 35.2%, in rural areas 30.3%. Youth unemployment is 18.8% against 13.1% for adults, and overall unemployment rose from 13.4% to 15.6% in one year. Labour underutilisation is 61.8%: more than half of the extended labour force is unemployed, under-employed or available but not searching. Jobs are concentrated: 48.9% of workers are in elementary occupations and 34.2% in agriculture, so most young people have no guidance on other paths that exist near them. Education alone does not solve it: unemployment is 22.8% among upper-secondary leavers, higher than among primary leavers (15.2%). The missing piece is personal fit plus local demand: people choose paths without knowing what they are good at or where jobs are growing.

2. Our solution A free, Kinyarwanda-first mobile tool. Short taps and mini-games (focus, reasoning, memory) build an interest and thinking-skills profile. The engine ranks 15 paths using that profile AND NISR labour-market demand (sector share and job change from the LFS), then gives a first 20-minute step. An aggregated, anonymised dashboard shows talent supply against demand for planners.

3. Why it can work 92.2% of 16-30 year-olds have basic digital skills (LFS Table B.4): a phone-first tool reaches the group at risk. Engine runs in the browser: works with weak data, no per-user server cost, 6 languages, privacy-first, under-18 safe.

4. Alignment NST2 / Vision 2050: Vision 2050 and NST2 youth employment and skills priorities. Track 3 Open Innovation.

5. Honest limits Profiles are starting hypotheses, not IQ or clinical tests. Path-to-sector mapping is our assumption and is documented in data/processed/demand_by_path.csv. District-level demand needs LFS microdata access (requested). Pilot with 30+ users planned to calibrate weights.

BUSINESS MODEL (draft)

Principle: free for the young person; paid by the institutions that gain from better matching.

Users (free forever)

- Secondary and TVET students, unemployed youth, school leavers, people changing work.

Customers / payers (after the hackathon)

1. TVET schools, secondary schools, universities: guidance tool + cohort reports (subscription per school or per term).
2. NGOs, youth programmes, development partners: programme targeting + outcome tracking (project-based fee / grant-funded licence).
3. Districts / government agencies (WDA, MINEDUC, MIFOTRA, NISR): anonymised dashboard of talent supply vs. labour demand by district (service agreement).
4. Employers / training providers: sponsored "pathway" listings, clearly labelled, never mixed into the ranking.

Revenue streams

- School and programme licences; dashboard service contracts; grants; sponsored training listings (labelled).

Costs (low by design)

- Engine runs in the browser (near-zero per-user server cost); static hosting; small API + database for saved sessions.
- Main real cost: native-reviewed translations and keeping the demand data updated from NISR releases.

Why it can last

- Data-light PWA fits low-income phones and weak networks; Kinyarwanda first.
- Each completed session adds anonymised aggregate evidence (consent-based) that improves matching and feeds the dashboard.

Rules we keep

- Ranking is never for sale. Contacts only with consent; under-18 needs a guardian. No "pay first to get a job" paths.

IP note

- Hackathon entry assigns IP of the submitted work to NISR; this repository contains only the hackathon work.

BEST SIDES TO PITCH (map to the 5 scoring criteria, 20 pts each)

1. Problem understanding and relevance

- Youth skills-to-jobs mismatch is a national priority. Open with one NISR number (youth unemployment or NEET) and one district example.
- Say the NST2 / Vision 2050 pillar by name.

2. Data use and methodology (weakest today - fix first)

- Show the demand layer: LFS/EICV/Census values by district and sector replacing hand-set market weights.
- Show the method in one slide: interest + style + motives + evidence + local demand -> ranked paths with confidence.
- Show data cleaning steps and sources table (dataset, year, table, what it feeds).

3. Tech innovation

- Adaptive question selection (asks what it is least sure about), stops by rule, bootstrap stability for confidence.
- Detects yes-saying, rushing, inconsistency; deterministic and testable (simulate.ts, lint-content.ts, CI).
- Add 3 mini-games (focus, reasoning, working memory) feeding the same engine as evidence.

4. Usability and design

- 8 context taps, icons, no typing; 6 languages incl. Kinyarwanda; offline PWA; works with no data; read-aloud planned.
- Live demo on a cheap Android phone in Kinyarwanda. This is your strongest moment.

5. Tangible impact

- Policy dashboard: anonymised talent supply vs. demand by district.
- Pilot plan: 30+ users in Kinyarwanda, then a school/TVET; measure step completion rate and "liked it" rate.
- Clear users, payers, and a first pilot partner idea (a TVET or youth programme).

Demo order (3-5 min)

1. one NISR fact -> 2) phone demo in Kinyarwanda -> 3) result with "because jobs in your district..." -> 4) first step -> 5) dashboard -> 6) honest limits + pilot plan.

Do not claim

- "scientifically validated", "measures IQ", "diagnoses stress". Say: interest and thinking-skills profile, pilot in progress.

VERIFIED FACTS (NISR LFS Q3 2026, collected 1-31 Aug 2026) - every line checked against the downloaded files

source files: data/raw/publications/LFS_2026Q3_summary_ENG.pdf, LFS_August_2026_report.pdf, LFS_2026Q3_tables.xlsx

- Working-age population (16+): 8,808,239 | employed 4,752,916 | unemployed 877,716 | outside labour force 3,177,607
- Labour force participation 63.9% (65.8% in Q3 2025) | employment-to-population 54% (57%)
- Unemployment rate 15.6% (+2.2 pts vs Q3 2025) | female 18.3% | male 13.3% | youth (16-30) 18.8% | adults 13.1% | rural 16.0% | urban 14.8%
- Youth NEET 27.5% = 1,038,707 young people | female 35.2% | male 19.6% | rural 30.3% | urban 22.1%
- Composite labour underutilisation 61.8% (report p24) | youth 63.1% (Table 4)
- Unemployment by education (Table 6): upper secondary 22.8% | university 12.4% | lower secondary 17.8%
- Employment by sector (Table 7): agriculture 34.2% | trade 15.4% | construction 11.9% | transport 7.7% | manufacturing 6.2%
- Occupations (Table 9): elementary 48.9% | service and sales 19.9% | craft trades 9.7% | professionals 6.0%
- Basic digital skills, age 16-30: 92.2% (Table B.4) - supports a phone-first tool
- Median monthly earnings at main job: 52,000 (youth table, Q3 2026)

DO NOT QUOTE data/raw/worldbank values for unemployment (modeled ILO 16.4%): use NISR numbers above. Where an LFS 2022 number is used, say "2022". District-level LFS needs microdata login (microdata.statistics.gov.rw study 114): register and request access.

COOKER — algorithm (pseudocode) + project structure

v0.1 · 2026-10-20 · languages: rw · en · fr · ar · sw · lg

Goal: help a person find what they can do, how to start, and why — by small tested steps, not by guessing a job title.

```
PART A  algorithm            A0 rules · A1 vocabulary · A2 data shapes · A3 main flow · A4 setup stages
                             A5 joy map · A6 state · A7 adaptive cards · A8 reality check · A9 matching
                             A10 step planner · A11 follow-up loop · A12 learning · A13 UX logic
PART B  structure            Vercel (web) + Render (api) · repo · endpoints · tables · deploy · free-tier
facts
PART C  v0.1 scope · tests · honest limits
```

PART A — ALGORITHM

A0. Rules the logic never breaks

```
R1  LANGUAGE-FREE CORE   engine sees only ids and numbers. All words live in i18n files. (language barrier
solved at the root)
R2  ONE TAP              every question answerable in ≤3 s by tapping. No typing, except optional contact at
the end.
R3  BEHAVIOR > WISH      what you DO / DID counts more than what you WANT to be (influencer filter).
R4  NO FAST PREDICTION   output = ranked hypotheses + confidence + cheap experiments. The real answer comes
from follow-up.
R5  EXPLAINABLE          every result says "because you said ..." in the user's language. No black box.
R6  SMALL STEPS          first step ≤ 20 min/day, free, in their language, fitted to their personality
(talker vs quiet).
R7  REVIEWED TEXT ONLY   never show an item/step/path whose text is not native-reviewed in the chosen
language.
R8  TWO LANES            CASH lane (can start now with what you have) + PASSION lane (grow into it).
R9  SAFE BY DEFAULT      under-18: no contact stored without guardian. No "pay first to get a job" paths.
Scam warning at earning stage.
R10 DETERMINISTIC        same answers → same result (seeded randomness). Testable.
```

A1. Vocabulary — what the engine measures

```
Every dimension d has:  mu[d] ∈ [-1,+1]  (best guess)   var[d] ∈ (0,1]  (how unsure; 1 = know nothing)

INTEREST  (Holland RIASEC)
   R  hands / tools / outdoors / machines
   I  figuring out / learning / researching
```

```
   A  making / creating / expressing
   S  helping / teaching / caring
   E  leading / selling / convincing
   C  organizing / numbers / records

STYLE  (how they like to work)           -1 pole                  +1 pole
   social_energy                          quiet, solo, few people  talks a lot, gains energy from people
   structure                              free, changing           routine, clear rules
   pace                                   slow, steady             fast, pressure
   physical                               sitting                  body-active
   outdoors                               inside                   outside
   risk                                   secure                   uncertain, try new
   visibility                             behind the scenes        seen, performing
   independence                           team / employer          own boss

MOTIVES   (Self-Determination + values)    autonomy · mastery · helping · security · status ·
community_impact
EFFICACY  efficacy[domain] ∈ [0,1]         "can I learn this?" (Bandura) — raised only by DOING (follow-up
outcomes)
CAPITAL   (context, NOT personality)       time_hours_week · money(0-3) · data(0-3) · device(0-3) ·
education(0-3)
                                           · languages[] · location{district, urban|rural|abroad} ·
support(0-3) · dependents
STAGE     age_band · situation             → stage_policy() changes weights, step size, wording
```

A2. Data shapes (content lives in files, never in code)

```
ITEM {  id, kind: CARD | PICK2 | TILE | TRAP,
        group: ENJOY | PRACTICE | STYLE | MOTIVE | EFFICACY | ASPIRE | DEALBREAKER,
        loads: { dim: weight ∈ [-1,+1] },          # direction of a YES answer. A reverse twin simply has
opposite signs.
        twin_of: item_id | null,                   # reverse-keyed partner → detects yes-saying +
inconsistency
        excludes: [path_id] | null,                # DEALBREAKER only
        icon, text_key, audio_key?, status{lang: draft|reviewed} }

PATH {  id, cluster, tags[],
        interest_vec: {R,I,A,S,E,C},
        style: { dim: (target ∈[-1,1], importance ∈[0,1]) },     # what the path DEMANDS
        motives: { m: weight },
        barrier: { money, data, education, tools, location, time }   # each 0..3
        cash_lane: bool,  bridge: path_id | null,                  # lower-barrier doorway into this path
        market: { region: demand 0..1, updated_on },               # YOU maintain this from real sources
        day_in_life: [item_id x3],  immersion: [list],             # films, radio, channels per language
(curated)
        ladder: [LEVEL L0..L7 → STEP[]] }

STEP {  id, level, type: WATCH|COPY|MAKE|SHOW|ASK|PORTFOLIO|EARN|LEARN,
        minutes, cost, needs{data, device}, social_level ∈ {0 alone, 1 by message, 2 face to face},
        days_expected, text_key, done: ONE_TAP }

TILE (joy map) { id, icon, loads{dim:w}, tags[] }     # e.g. football, cooking, fixing phones, drawing,
chatting, farming ...
```

A3. Main flow

```
FUNCTION run_cooker():
    lang    = stage_language()                    # first screen, always
    consent = stage_consent(lang)                 # what we store, why, optional contact
    ctx     = stage_context(lang)                 # ≤ 8 taps
    policy  = stage_policy(ctx.age_band, ctx.situation)
    state   = init_state(ctx, policy)             # mu=0, var=1 for all dims (flat, honest start)
    state   = stage_joy_map(state, lang)          # revealed behavior: what you do for hours
    state   = stage_adaptive_cards(state, lang, policy)   # yes/no + forced pairs, ends by RULE not by
count
    state   = finalize_state(state)               # acquiescence correction, consistency flags
    state   = stage_reality_check(state, lang)    # day-in-life + borrowed-dream filter
    result  = stage_match(state, policy)
    show_result(result, lang)                     # lanes, reasons, first 3 steps
    stage_followup_setup(state, result, consent)
    RETURN result
```

A4. Setup stages

```
FUNCTION stage_language():
    show 6 big buttons, each written in its OWN language/script:
        Ikinyarwanda · English · Français · العربية · Kiswahili · Luganda       # no flags (flag ≠ language)
    preselect a guess from browser locale / last visit, but the user must TAP (this tap also unlocks audio)
    IF lang == ar: ui.dir = RTL
    ask optional 2nd language ("I also speak ...")  → used as FALLBACK and for finding learning resources
    ask once: "read aloud?" (speaker icon)  → audio_on
    lang_fallback = second_lang OR en
    # R7: any text without reviewed status in lang is hidden; engine skips that item/step (beta flag may
show fallback with a marker)

FUNCTION stage_consent(lang):
    show 3 icon+sentence lines:  what we ask · why · "may we remind you about your first step?"
    consent = { analytics: bool, contact: bool, guardian: bool|null, text_version, timestamp }
    IF age_band == UNDER18 AND contact: require guardian consent flag before ANY contact is stored      #
R9
    RETURN consent                                  # everything works with contact = false

FUNCTION stage_context(lang):       # all pick-one with icons, skippable
    ask: age_band (UNDER18 | 18-24 | 25-39 | 40+)
         situation (SCHOOL | FINISHED_NO_WORK | WORKING | OWN_BUSINESS | FARMING | ABROAD_OR_RETURNED |
OTHER)
         where_from (province/district list | country) , area (URBAN | RURAL)
         education (NONE_PRIMARY | SECONDARY | TVET | UNIVERSITY)
         free_time (≤3h | 3-10h | 10h+ per week)
         can_spend_to_start (NOTHING | A_LITTLE | SOME)            # phrased with no shame
         phone_access (NO_DATA | SOMETIMES | YES) , device (BASIC | SMARTPHONE | COMPUTER)
         someone_to_help (NO | MAYBE | YES) , dependents (NO | YES)
    RETURN ctx

FUNCTION stage_policy(age_band, situation):          # psychology of life stages, as weights + step rules
    W = { interest:.30, style:.25, motive:.15, evidence:.20, reality:.10 }
    MATCH age_band:
```

```
      UNDER18 : # identity exploration (Erikson), career curiosity (Super)
                W.interest += .10 ; W.evidence -= .10 ; show top 5 as "things to try", not "your job"
                step_rules = FREE_ONLY, SCHOOL_COMPATIBLE ; contact needs guardian
      18-24   : # exploration → trial
                default W ; allow apprenticeship / short course at L7
      25-39   : # establishment: income pressure
                W.reality += .05 ; feasibility weight ↑ ; prefer BRIDGE paths from existing skills
      40+     : # maintenance / reinvention: experience counts
                W.evidence += .10 ; W.interest -= .10 ; avoid "start from zero" steps ; favour
teach/train/small-business paths
    MATCH situation:
      FINISHED_NO_WORK | dependents : cash lane shown FIRST
      WORKING                       : "side-start" steps ≤ 15 min
      FARMING                       : paths built on land/animals/produce first (agro-processing, selling,
tech for farmers)
      ABROAD_OR_RETURNED            : add remote / language / diaspora-bridge paths
      SCHOOL                        : steps fit school week ; weekends for bigger steps
    RETURN { W: normalize(W), step_rules, display_k, cash_first }
```

A5. Joy map — revealed behavior (the most honest signal)

```
FUNCTION stage_joy_map(state, lang):
    picked = user picks ≤ 5 tiles from TILE bank (≈24 icons)   "what do you do and forget the time?"
    FOR t in picked:
        freq   = tap3(t, [DAILY, WEEKLY, SOMETIMES])
        output = tap3(t, [DID_IT_FOR_OTHERS, MADE_SOMETHING, ONLY_FOR_ME])     # social proof / skill trace
        ev[t]  = 0.5·FREQ_W[freq] + 0.5·OUT_W[output]                          # FREQ_W: 1 / .6 / .3 ;
OUT_W: 1 / .7 / .3
        FOR (d,w) in t.loads:  add_evidence(state, d, signal=sign(w), weight=|w|·(0.5+ev[t]),
source=PRACTICE)
        state.practiced[t] = ev[t]
    helped = user picks ≤ 3 tiles "what do people ask YOU for help with?"       # competence seen by others
    FOR t in helped: add_evidence(state, t.loads, +1, weight .8, source=PRACTICE) ; state.practiced[t] =
max(.., .7)
    state.candidates = { p in PATHS : p.tags ∩ (picked ∪ helped).tags ≠ ∅ }  ∪ top paths by current
interest   # don't test all paths
    RETURN state
```

A6. State and evidence update

```
RELIABILITY = { OUTCOME:1.0, PRACTICE:1.0, FORCED:0.9, ENJOY:0.8, STYLE:0.7, MOTIVE:0.7, EFFICACY:0.7,
ASPIRE:0.3 }

FUNCTION add_evidence(state, d, signal ∈ [-1,+1], weight, source, quality=1.0):
    p_obs = weight · RELIABILITY[source] · quality              # precision of this observation
    p_old = 1 / state.var[d]
    p_new = p_old + p_obs
    state.mu[d]  = (state.mu[d]·p_old + signal·p_obs) / p_new     # precision-weighted average
    state.var[d] = 1 / p_new                                      # every answer makes us surer

FUNCTION mu_eff(state, d):  RETURN state.mu[d] · (1 − min(state.var[d], 1))   # unsure → pulled to neutral,
never over-claims
```

A7. Adaptive cards — questions chosen by what we still don't know

```
ANSWERS = { YES:+1, SOMETIMES:+0.3, NO:-1, SKIP:0 }        # SKIP = "not sure / don't know this" → NOT
counted as no

FUNCTION stage_adaptive_cards(state, lang, policy):
    asked = []
    WHILE NOT stop_rule(state, asked):
        item = pick_next_item(state, asked, lang)
        (answer, ms) = show(item)                                  # tap YES / SOMETIMES / NO / SKIP, or
choose A/B
        q = assess_quality(item, answer, ms, asked, state)
        apply_response(state, item, answer, q)
        asked.append(item)
        update_pot(state)                                          # progress = real information gained (A13)
    RETURN state

FUNCTION apply_response(state, item, answer, q):
    IF answer == SKIP:  state.skips[item.group]++ ; RETURN
    IF item.kind == PICK2:                                         # forced choice: immune to yes-saying
        (win, lose) = chosen, other
        FOR (d,w) in win.loads:   add_evidence(state, d, +sign(w), |w|, FORCED, q)
        FOR (d,w) in lose.loads:  add_evidence(state, d, -sign(w), |w|, FORCED, q)
        RETURN
    s = ANSWERS[answer]
    FOR (d,w) in item.loads:  add_evidence(state, d, s·sign(w), |w|, item.group, q)
    state.raw_cards.append((item, s))                              # kept for acquiescence correction
    IF item.group == DEALBREAKER AND answer == NO:  state.hard_exclude ∪= item.excludes
    IF item.group == ASPIRE:  state.aspirations.add({ path: item.path, answered: answer })

FUNCTION pick_next_item(state, asked, lang):
    leaders = top_paths(state, k=6)                                # leaders AND near-leaders
    cands   = [ i in ITEM_BANK : i ∉ asked AND reviewed(i, lang) AND allowed_now(i) ]
    FOR i in cands:
        # usefulness = touches dimensions we are UNSURE about, where the leaders DISAGREE
        sep[i]   = Σ_d |i.loads[d]| · state.var[d] · spread(leaders, d)         # spread = max − min of
leaders' target on d
        score[i] = sep[i] − 0.4·same_dim_recently(i, last 3) + 0.2·fun(i) + group_balance_bonus(i, asked)
    ENFORCE (override score):
        cards 1-3        : easy, fun, non-sensitive warm-up, one per RIASEC-ish theme
        every 5th card   : a PICK2 forced pair
        every ~7th card  : the reverse TWIN of an earlier answered item   (yes-saying + consistency check)
        at ~40% progress : one TRAP item (obvious answer, e.g. "I never eat food" → NO expected)
        rhythm           : easy → thinking → easy (never 3 heavy cards in a row)
        DEALBREAKER      : only when a leader path demands that dimension (else skip — save time)
        ASPIRE items     : only after the joy map, only if user picked a "want to be" tile
    RETURN argmax(score)

FUNCTION stop_rule(state, asked):                          # ends by RULE, not by fixed count
    n = len(asked)
    IF n < 14: RETURN false                                # minimum evidence
    IF n ≥ 30: RETURN true                                 # hard cap ≈ 3 minutes
    IF fatigue(asked):  state.confidence_cap = LOW ; RETURN true
    stab = bootstrap_stability(state, B=60)                # see A9
    key  = dims with importance ≥ .5 in the top 6 paths
```

```
    RETURN stab.top3_overlap ≥ 0.80 AND all(state.var[d] < 0.25 for d in key)

FUNCTION fatigue(asked):   RETURN last 8 answers identical OR median(ms of last 6) < 400 OR user pressed
"later"

FUNCTION assess_quality(item, answer, ms, asked, state):
    q = 1.0
    IF ms < 400:                          q *= 0.5          # didn't read (tune with pilot data)
    IF last 8 answers identical:          q *= 0.6 ; state.next_forced_pair = true
    IF item.kind == TRAP AND wrong:       state.flags += CARELESS ; state.q_global *= 0.7
    IF item.twin_of in asked AND |answer − earlier answer·(−1)| > 1.2:   state.flags +=
INCONSISTENT(item.dims) ; q *= 0.7
    RETURN q

FUNCTION finalize_state(state):
    # acquiescence: do they say YES to everything, even to opposite statements?
    acq = mean(s for (item, s) in state.raw_cards)          # in a balanced bank, honest answers average ≈
0
    IF acq > +0.35 (or < −0.35):
        state.flags += YES_SAYER (or NO_SAYER)
        REPLAY all raw_cards with s' = s − 0.8·acq          # remove the bias, keep the difference between
items
        raise FORCED weight ×1.5 ; state.confidence_cap = MEDIUM
    IF state.q_global < 0.6: state.confidence_cap = LOW ; invite redo later
    RETURN state
```

A8. Reality check + borrowed-dream filter (online-influencer problem)

```
FUNCTION stage_reality_check(state, lang):
    preview = top_paths(state, k=6)
    FOR p in preview[0:4]:
        FOR it in p.day_in_life:                            # 3 cards: "Would you be OK doing {daily
reality} for 4 hours?"
            ans = tap(it)                                   # YES / NO
            state.reality[p] += (ans == YES) ? 1/3 : 0
    FOR a in state.aspirations:                             # the user said "I want to be X"
        origin = tap6(a.path, [ I_TRIED_IT | SOMEONE_CLOSE_DOES_IT | SAW_ONLINE | FAMILY_WANTS | FOR_MONEY
| I_JUST_LIKE_IT ])
        evid   = evidence_fit(state, a.path)
        IF origin == SAW_ONLINE AND evid < 0.3:
              a.status = BORROWED_DREAM         # not hidden, not blamed — tested first
              state.taste_tests.add(a.path)     # 5-day TASTE TEST (L0 + L1 steps) + real day-in-life,
before any bigger step
              state.weight_cap[a.path] = 0.6    # cannot become Top-3 until it survives the taste test
        ELIF origin == FAMILY_WANTS:
              a.status = EXTERNAL               # ask 1 card: "and do YOU want it?" ; show side by side
with own-fit paths
        ELSE: a.status = OWNED
    RETURN state
```

A9. Matching

```
FUNCTION stage_match(state, policy):
    cand = [ p in PATHS : p ∉ state.hard_exclude ]
    FOR p in cand:
        fi = (cosine( mu_eff(RIASEC), p.interest_vec ) + 1) / 2                                  # 0..1
        fs = 1 − Σ_d imp_d·|mu_eff(d) − target_d|/2  /  Σ_d imp_d      over d in p.style         # 0..1
        fm = ( Σ_m p.motives[m]·mu_eff(m) / Σ|p.motives| + 1 ) / 2
        fe = clip( Σ state.practiced[t] for t in p.tags /2  +  0.3·n_liked(p) − 0.4·n_disliked(p), 0, 1 )
# includes follow-up OUTCOMES
        fr = state.reality[p] if asked else 0.5
        W    = policy.W                                                    # from stage_policy (sums
to 1)
        raw  = W.interest·fi + W.style·fs + W.motive·fm + W.evidence·fe + W.reality·fr
        raw  = min(raw, state.weight_cap[p] or 1)                          # borrowed dreams
        unc  = mean( state.var[d] for d in dims_of(p) )
        adj  = raw − 0.3·unc                                               # unsure → lower, don't
over-claim
        feas = feasibility(state.ctx, p)
        p.score = adj · (0.5 + 0.5·feas)
        p.parts = {fi, fs, fm, fe, fr, feas}                               # kept for explanations

    ranked = sort(cand, by score, desc)
    FOR p in ranked WHERE feas(p) < 0.4 AND p.bridge:  replace p by BRIDGE(p) labeled "start here → grow to
p"

    top    = MMR_select(ranked, k = policy.display_k or 3, lam = 0.7)          # relevant AND different
from each other
    surprise = argmax over ranked[3..15] of ( 0.5·norm(score) + 0.5·unc )       # adjacent path that
teaches us most
    cash   = best p in cand where p.cash_lane AND feas ≥ 0.7 AND (fi+fs)/2 ≥ 0.5
    conf   = bootstrap_stability(state, B=100) → per path: HIGH ≥ .80 · MEDIUM ≥ .50 · else LOW, capped by
state.confidence_cap
    RETURN { top, surprise, cash, taste_tests: state.taste_tests, conf, flags: state.flags,
             reasons: { p: reasons(state, p) for p in top ∪ {surprise, cash} },
             strengths: top3_strengths(state) }

FUNCTION feasibility(ctx, p):                       # soft penalty, never a wall
    f = 1
    FOR b in [money, data, education, tools, location, time]:
        gap = max(0, p.barrier[b] − ctx.capital[b])            # 0..3
        f  *= (1 − 0.25·gap)
    RETURN max(f, 0.05)

FUNCTION MMR_select(ranked, k, lam):
    S = []
    WHILE len(S) < k:
        S.add( argmax_{p ∉ S} [ lam·norm(score[p]) − (1−lam)·max_{q∈S} similarity(p,q) ] )   # similarity =
cosine(vec) + same-cluster bonus
    RETURN S

FUNCTION bootstrap_stability(state, B):             # how fragile is the answer?
    FOR b in 1..B (seeded):
        mu_b[d] = Normal( state.mu[d], sqrt(state.var[d]) )      # re-draw what we're unsure about
        rank_b  = rank paths using mu_b
        record top3_b, top5_b
    top3_overlap = average overlap of top3_b with the main top3
```

```
    stability[p] = fraction of runs where p ∈ top5_b
    RETURN { top3_overlap, stability }

FUNCTION reasons(state, p):                          # explainable output, template-based
    contrib = { d: mu_eff(d)·p.weight_on(d) for d in dims } ∪ { tile t: state.practiced[t] for t in p.tags
}
    why   = top 3 positive contributors → i18n keys "because you {said/do/like} ..."
    watch = largest negative contributor OR largest barrier gap → key + workaround ("no data? use
radio/offline version")
    RETURN { why[3], watch[1] }

FUNCTION top3_strengths(state):                      # people-friendly, no labels/types, no diagnoses
    dims = top 3 by |mu_eff| ; text = i18n["strength."+dim+"."+pole]       # e.g. "you focus well alone",
"you explain things simply"
```

A10. Step planner — "don't go study", break it into steps

```
LADDER (same skeleton for every path; each path fills its own steps)
  L0 WATCH/LISTEN   consume the field daily with ONE noticing question          (e.g. 1–2
films/documentaries/radio/YouTube per day for a week)
  L1 COPY           redo one tiny piece yourself, imitating                      (retell a news item in
your words; cook the same dish; fix the same fault)
  L2 MAKE SMALL     make 1 small thing of your own (≤30 min), no audience
  L3 SHOW           show 3–5 people you trust, ask ONE question: "what was clear / not clear?"
  L4 ASK            message or meet someone who does it (script given in their language)
  L5 PORTFOLIO      5–10 pieces over 4–6 weeks, a streak
  L6 EARN           first paid / real use (micro-job, local gig, volunteer for a real organisation) + SCAM
CHECK (R9)
  L7 LEARN (opt.)   course / apprenticeship / scholarship — only now, with evidence they will finish

FUNCTION plan_steps(state, p, policy):
    start   = highest level whose steps are already evidenced by state.practiced   # skip WATCH if they
already do it
    minutes = clamp( ctx.time_hours_week·60/7 · 0.25, 10, 45 )                      # a quarter of free
time, never more than 45 min
    plan = []
    FOR level in ladder[start:]:
        pool = [ s in level.steps : s.needs ≤ ctx(data, device)  AND  s.cost ≤ ctx.money  AND  reviewed(s,
lang)
                 AND  s.minutes ≤ minutes·(1 + efficacy(p.domain)) ]
        pool = filter(pool, policy.step_rules)                                      # UNDER18: free +
school-compatible
        # personality fit
        IF mu_eff(social_energy) < -0.3:  prefer s.social_level ≤ 1   (message instead of meeting, write
instead of call)
        IF mu_eff(social_energy) > +0.3:  prefer s.social_level = 2   (talk to 3 people, join a group,
perform)
        IF efficacy(p.domain) < 0.4:      lengthen L0–L1 (more watching/copying before making)
        IF mu_eff(structure) > +0.3:      schedule "same time every day" ELSE "any 5 days this week"
        plan.append( best(pool) )
    RETURN plan[0:3] visible now ; the rest locked until check-ins confirm (level-up)

EXAMPLE  p = journalism / storytelling  (user: talker, smartphone, low money)
   L0  Daily 20 min: listen to one radio news item or watch one short report. Question: "what did the
```

```
reporter ask first?"  + 1–2 films/day on weekends (curated list)
   L1  Retell one thing that happened in your village in 5 sentences (voice note to yourself)
   L2  Record a 60-second voice report about a real local issue
   L3  Send it to 5 people; ask: "was it clear?"
   L4  Message one local radio/online reporter with the given script (quiet user: message; talker: call or
visit)
   L5  10 reports in 6 weeks → small portfolio page / WhatsApp channel
   L6  Offer one story to a local outlet or community radio
   L7  Only now: journalism course/scholarship options
```

A11. Follow-up loop — where the real learning happens

```
FUNCTION stage_followup_setup(state, result, consent):
    IF NOT consent.contact: RETURN
    IF age_band == UNDER18 AND NOT consent.guardian: RETURN
    FOR each chosen path's first visible step s:
        schedule(user, due = now + s.days_expected, channel = user.channel (EMAIL v0.1), lang = user.lang)
    reminders after: D+2 · D+7 · D+21 · D+45 · D+90 (adjusted by s.days_expected)
    max_unanswered = 3 → pause politely ; every message has a 1-tap STOP link

FUNCTION on_followup_tap(user, step, answer, reason):           # opened from a signed magic link, no login
    MATCH answer:
      DONE_LIKED:     add_evidence(step dims, +1, source=OUTCOME) ; efficacy[domain] += 0.1 ; unlock next
level
      DONE_NEUTRAL:   unlock next level (small evidence)
      DONE_DISLIKED:  ask WHY (1 tap):
                        TOO_BORING → interest dims ↓       TOO_SOCIAL → social_energy ↓       TOO_LONELY →
social_energy ↑
                        TOO_HARD   → efficacy ↓, smaller steps       NOT_FOR_ME → path weight ↓
(n_disliked++)
                      re-run stage_match with updated state → send "new recipe" (next-best path)
      NOT_YET:        ask BARRIER (1 tap):
                        NO_TIME  → shrink to 10 min          NO_DATA  → offline version (radio, library,
ask someone)
                        NO_TOOLS → bridge step               FORGOT   → change time + channel
                        AFRAID   → lower social/risk step + "do it with a friend"     NO_HELP → (later)
mentor/group matching
      NOT_FOR_ME:     hard down-weight the path ; propose runner-up
    log OUTCOME event (user, path, step, answer, reason, ts)         # your most valuable data
    # difference matters: DISLIKED = mismatch (change path) ; NOT_YET = barrier (change the step)
```

A12. Learning from everyone (when data exists)

```
NIGHTLY JOB:
    FOR each (profile_cluster, path):      enjoy_rate, completion_rate  → shrink toward prior (use only if
n ≥ 30)
    FOR each (item, lang):                 DIF check — does the item behave differently by language? → flag
for translator review
    FOR each (step, social_level, minutes, lang):  completion_rate → Thompson sampling picks which first
step to try next time
    REGRESS: which fit parts (fi, fs, fm, fe, fr) predict DONE_LIKED → propose new W   (ADMIN APPROVES;
```

```
never auto-change catalog)
    REPORT: skip-rate per item, median ms per item, drop-off card → rewrite/remove the bad cards
```

A13. UX logic (people-friendly, not boring)

```
THEME   "Cooker" = cooking pot. Answers drop ingredients into the pot. Result = "ready to taste".
CARD    one idea, ≤12 words, icon, 3 big buttons (✓ yes · ~ sometimes · ✕ no) + swipe. Skip is a small
link. RTL mirrors layout for ar.
POT     progress = max(previous, 1 − mean(var[d] for key dims)) → bar is real information gained, never
goes backwards
HALFWAY  ~card 12: "Half cooked!" 2-second break, then continue ; offer "continue later" (state autosaved
locally)
AUDIO   WebAudio synth, NO audio files (data-light). Starts only after the language tap (autoplay rule).
        bpm≈92, pentatonic scale, bass + hat loop
        YES = rising note · NO = lower note · SOMETIMES = middle ; streak of quick answers adds a layer
(hat → clap)
        mute always visible, remembered ; OFF by default if Save-Data/battery-saver ; haptic tick on mobile
READ-ALOUD  human-recorded short clips per card (small files) when available ; browser TTS for rw/lg may be
missing → verify before relying on it
RESULT  3 cards + 1 surprise + 1 cash lane. Each card: WHY (3 reasons) · FIRST 3 STEPS · WATCH OUT (1
honest thing) · "Try it today (≤20 min)"
```

PART B — STRUCTURE (frontend on Vercel, backend on Render)

B1. Core decision: the engine runs in the browser

```
BROWSER (Vercel, static)    survey + matching run locally → ZERO api calls while answering
                            → no Render cold start in the middle of a question, works on weak network,
costs nothing per user
                            → progress autosaved (IndexedDB/localStorage) so a phone dying doesn't lose it
API (Render)                only 4 moments:  (1) save finished session  (2) follow-up taps  (3) opt-
out/delete  (4) admin/content
                            → server RE-RUNS the same engine on the submitted answers; never trusts the
client's result
Pre-warm                    on the language tap, web fires GET /health (fire-and-forget) so the API is
awake when the result is saved
Save                        result shows instantly from local compute; saving happens in background with
retry queue
```

B2. Repo (one monorepo — easiest to grow alone)

```
cooker/
├─ apps/
│  ├─ web/                        → VERCEL   (Vite + React + TypeScript, PWA, static)
```

```
│  │  ├─ src/screens/            Language · Consent · Context · JoyMap · Cards · Result · Steps
│  │  ├─ src/audio/beat.ts       WebAudio generator
│  │  ├─ src/i18n/               lazy-load packages/content/i18n/<lang>.json ; RTL switch for ar
│  │  └─ src/store/              local progress (IndexedDB), retry queue
│  └─ api/                       → RENDER   (Node + Fastify + TypeScript + Postgres)
│     ├─ src/routes/             sessions · followup · optout · admin · internal
│     ├─ src/services/           mailer · tokens (signed magic links) · crypto (encrypt contact) ·
scheduler
│     └─ src/db/migrations/
├─ packages/
│  ├─ engine/                    PURE functions, no I/O: init_state · add_evidence · pick_next_item ·
stop_rule
│  │                             · finalize_state · stage_match · plan_steps · on_followup_tap
│  ├─ content/                   items.yaml · tiles.yaml · paths.yaml · steps.yaml ·
i18n/{rw,en,fr,ar,sw,lg}.json
│  └─ shared/                    zod schemas · types · constants (dims, groups, answer values)
├─ tools/
│  ├─ simulate.ts                personas → engine → print results (sanity tests, Part C)
│  ├─ lint-content.ts            every key in all 6 langs? status=reviewed? text ≤ 12 words? loads in
range?
│  └─ dif-report.ts              (later) item behaves differently by language?
├─ .github/workflows/           ci.yml (typecheck + lint-content + simulate) · followups.yml (daily call
to API)
└─ README.md
```

Why TypeScript for the API: the same `packages/engine` runs in browser AND server → one logic, no drift. (If you prefer Go for the API, the engine would run only on the server; you lose the offline/cold-start benefit.)

B3. API (Render)

```
GET   /health                                warm-up, no data
POST  /v1/sessions        {lang, ctx, consent, answers[], tiles[], engine_version, content_version}
                          → server re-runs engine → stores → returns {result_token}
GET   /v1/result/:token                      re-open a result (token = unguessable, no login)
POST  /v1/followup/:token {answer, reason?} from the email's 1-tap links → on_followup_tap()
POST  /v1/optout/:token                      stop messages ; DELETE /v1/me/:token → erase everything
GET   /v1/content/version                    web checks if cached catalog is current
POST  /internal/run-followups                header X-Secret ; sends due emails (called daily by GitHub
Actions)
/admin/*                                     protected: paths, items, translation status, market data, W
approval
```

B4. Database (Postgres)

```
users       id uuid · lang · age_band · region · channel · contact_enc (encrypted) · consent jsonb {flags,
text_version, ts} · created_at
sessions    id · user_id? · ctx jsonb · answers jsonb · state jsonb · result jsonb · engine_version ·
content_version · quality_flags[] · created_at
plans       id · session_id · path_id · lane · level · step_id · unlocked_at
followups   id · user_id · step_id · due_at · sent_at · answer · reason · answered_at · attempt
events      id · user_id · type (OUTCOME|SKIP|DROP_OFF) · payload jsonb · ts          # analytics +
```

```
learning, no free text
content_versions  id · created_at · approved_by                                  # reproducible results
```

Anonymous by default: `user_id` exists only if consent.contact = true.

B5. Deploy checklist

```
Vercel   project root apps/web · build `vite build` · env: VITE_API_URL=https://api.<domain>
Render   Web Service apps/api (Node) · health check /health · env: DATABASE_URL, TOKEN_SECRET, CONTACT_KEY,
INTERNAL_SECRET,
         MAIL_API_KEY, ALLOWED_ORIGIN=https://<domain>  (CORS allow-list)
DNS      <domain> → Vercel · api.<domain> → Render
Email    send from your own domain with SPF/DKIM (otherwise it lands in spam) · pick a provider and CHECK
its free limits
Cron     06:00 UTC = 08:00 Kigali (CAT, UTC+2): GitHub Actions schedule → POST /internal/run-followups with
X-Secret
Privacy  contact encrypted at rest · no free text stored · consent text versioned · delete-me endpoint ·
under-18 rule
         Rwanda's personal-data law (Law 058/2021 — verify current obligations with a lawyer/regulator)
```

B6. Free-tier facts checked today (2026-10-20) and what they change

```
RENDER FREE web service  spins down after 15 min idle · ~1 min to wake · 750 free instance hours/month ·
filesystem wiped on restart
RENDER FREE Postgres     expires 30 days after creation (+14 days grace, then deleted) · 1 GB · no backups
RENDER FREE cron/workers not available on free
VERCEL HOBBY             free, but restricted to NON-COMMERCIAL personal use (fair-use guidelines)

→ engine in browser + pre-warm (B1) makes the 1-min wake harmless
→ do NOT keep real people's contacts on free Postgres: use a paid Render Postgres from day 1 (or an
external DB — check its terms)
→ follow-up scheduler = GitHub Actions calling the API (Render cron not free) ; the call also wakes the API
```

B7. Build order (solo)

```
1  packages/shared + content in ENGLISH only: 40 items, 24 tiles, 12–14 paths (ladders L0–L4 authored)
2  packages/engine + tools/simulate → Part C tests pass. NO UI yet.
3  apps/web: Language → Cards → Result (local only) + beat
4  Translate to all 6 languages + lint-content; a language is switched ON only when its text is reviewed
(R7)
5  apps/api + Postgres: save session, magic-link follow-up, email, GitHub Actions cron
6  Pilot in person, ~30 people (start in Kinyarwanda): watch hesitation, log ms/skip/drop-off per card
7  Use real outcomes to tune weights (A12)
```

PART C — v0.1 scope, tests, honest limits

C1. v0.1 content (small but complete — no holes)

```
Items     ≈36 CARD · 8 PICK2 · 3 TRAP · 6 DEALBREAKER · 24 TILES    (balanced: ≥1 reverse twin per STYLE dim)
Paths 14  storytelling/journalism · phone & electronics repair · tailoring & fashion · farming & agro-processing
          · cooking & catering · hair & beauty · teaching & tutoring · shop bookkeeping & sales · web/app building
          · photo/video/design · tourism guiding & hospitality · interpreting/translation (6 languages = advantage)
          · construction & welding · driving & logistics
          each with: interest_vec · style targets · barriers · day_in_life ×3 · ladder L0–L4 · bridge if barrier is high
Better 14 paths done well than 100 paths thin. Add paths only when someone asks for one and you can write its ladder.
```

Sample items (en text only; other languages after native review):

```
c001  "I like fixing things when they break."                    ENJOY  loads {R:+.8}
c002  "After many people, I feel tired and want quiet."          STYLE  loads {social_energy:-.9}
c003  "After many people, I feel full of energy."                 STYLE  loads {social_energy:+.9}  twin_of c002
c004  "I can sit alone for hours doing one thing."                STYLE  loads {social_energy:-.5, I:+.3}
c005  "I like making people laugh or telling stories."           ENJOY  loads {A:+.5, E:+.3, visibility:+.6}
c006  "I like to count money and keep records."                  ENJOY  loads {C:+.8}
c007  "I like to plan my day before it starts."                  STYLE  loads {structure:+.8}
c008  "I enjoy teaching someone how to do something."            ENJOY  loads {S:+.8}
c009  "I like convincing people to buy something."                ENJOY  loads {E:+.8, social_energy:+.4}
c010  "I like working outside, in sun or rain."                   STYLE  loads {outdoors:+.8, R:+.3}
f001  A "Build or repair something with my hands"  vs  B "Talk with people to solve their problem"  PICK2
t001  "I never eat food."  (expected NO)                         TRAP
d001  "Could you work in a loud place all day?"   (NO → excludes loud paths)   DEALBREAKER
```

C2. Sanity tests (must pass before any UI)

```
P1  quiet, draws, edits on phone, sits hours alone     → top3 ∋ photo/video/design | web/app | bookkeeping ; NOT sales/tourism as #1
P2  talks a lot, likes convincing, outdoors, risk high → top3 ∋ sales | tourism guiding ; NOT bookkeeping as #1
P3  hands-on, rural, no money, no data                 → farming/agro | construction | repair ; EVERY visible step works offline
P4  "I want to be a journalist", origin SAW_ONLINE, no practice → BORROWED_DREAM: shown as 5-day TASTE TEST, not rank 1
P5  answers YES to everything                          → YES_SAYER flag · confidence ≤ MEDIUM · result not extreme
P6  age UNDER18                                        → no contact stored without guardian · all steps free & school-compatible
P7  every card answered in < 300 ms                    → confidence LOW · invitation to redo later
P8  same answers twice                                 → identical result (R10)
INVARIANTS: Σ W = 1 · all scores ∈ [0,1] · no path/step without reviewed text in chosen lang is shown ·
```

```
every step respects ctx(data, money)
            · NOT_YET never lowers a path's score (barrier ≠ mismatch) · DONE_DISLIKED always does
```

C3. Honest limits

```
• Weights, loadings and thresholds above are EXPERT GUESSES until real outcomes exist. First ~100 users per
language = calibration, not proof.
  Do not call Cooker "scientifically validated". It gives starting hypotheses + experiments, not diagnoses.
• Translation is the real bottleneck (≈60 items + paths + steps × 6 languages). Native review for ar, sw,
lg needs people besides you.
  Short sentences + icons reduce risk; back-translate every item once.
• Rwanda market data (path.market) must be maintained from real sources and dated — never invented.
• Browser read-aloud for rw/lg may not exist → plan recorded audio.
• Emails need a domain + provider + consent text. Many users prefer WhatsApp/SMS: store optional phone now,
add that channel later.
```

Sources:

- Render free tier: https://render.com/docs/free
- Vercel Hobby plan: https://vercel.com/docs/plans/hobby